

PROJECT DEVELOPMENT & DEBUGGING REPORT

NetSentinel

Network Intelligence Platform

Repository	network-intelligence-platform
Development period covered	September 2026
Current status	Local end-to-end synthetic pipeline verified; real Windows-agent telemetry collection works; persistence of large cumulative byte counters is blocked by a PostgreSQL integer-width limitation

By- Arkaprava Roy

Table of Contents

Table of Contents	2
Executive Summary	3
1. Project Overview	4
2. Initial Build.....	4
3. First Environment Hurdle — Node.js Was Missing	6
4. Python Virtual Environment.....	6
5. Frontend Build Errors	6
6. npm Installation Warnings	7
7. Backend Test Database Failure	8
8. Docker Compose Failure — Wrong Directory	8
9. Docker Engine Not Running	9
10. First Successful Docker Deployment	9
11. Backend API Verification	10
12. Demo Data Integration Failure.....	10
13. Demo Database Fix.....	11
14. Synthetic Dashboard Verification.....	11
15. Windows Agent Registration.....	12
16. Windows Agent Configuration	12
17. Real Windows Device Registration.....	12
18. Real Windows Telemetry Collection Failure	12
19. Root Cause of the Real-Agent Failure	13
20. Correct Technical Direction for the Current Bug.....	13
21. Current Project Position	14
22. Current Known Warnings / Limitations.....	15
23. Why Cloud Deployment Has Not Happened Yet.....	16
24. Planned Next Steps	17
25. Development Philosophy Learned From the Build	19
26. Current Status Summary	20
27. Repository Hygiene Before Publishing	21
Final Milestone — Overall Status	21

Executive Summary

NetSentinel is a cloud-ready network intelligence and monitoring platform built to collect network and system telemetry from endpoint machines, persist that data centrally, analyze network health, detect anomalies, generate incidents, and present the results through a web dashboard. Development during September 2026 progressed from an initial multi-component scaffold through environment setup, build and test fixes, containerized deployment, synthetic-data validation, and real Windows-endpoint integration.

The platform's local, Dockerized deployment is fully operational, and the synthetic end-to-end pipeline — from telemetry ingestion through health scoring, anomaly detection, incident generation, and dashboard visualization — has been verified. A real Windows monitoring agent has also been successfully registered and is collecting live telemetry. The single outstanding blocker is a PostgreSQL schema limitation: the database's 32-bit INTEGER column cannot store cumulative network byte counters that exceed roughly 2.1 billion, a threshold real-world traffic volumes readily exceed. Resolving this requires a BIGINT migration that preserves existing data.

Cloud deployment has not yet begun by design: the project follows an incremental validation strategy in which each layer (build, tests, containers, synthetic data, dashboard, real telemetry, failure recovery) is proven before advancing to the next, so that environment, integration, and application issues are diagnosed independently rather than conflated. This report documents each development milestone, the issues encountered, their root causes, and the fixes applied, and closes with the current project status and the planned next steps toward production readiness.

Milestone	Status
Local Dockerized platform	Working
Synthetic end-to-end pipeline	Verified
Real Windows agent registration	Verified
Real Windows telemetry collection	Verified
Real Windows telemetry persistence	Blocked (BIGINT schema fix required)
Cloud deployment	Not started
Production hardening	Pending

1. Project Overview

NetSentinel is a cloud-ready network intelligence and monitoring platform designed to collect network and system telemetry from endpoint machines, persist the measurements centrally, analyze network health, detect anomalies, generate incidents, and present the results through a web dashboard.

The project combines the following components:

- Python / FastAPI for the backend API
- PostgreSQL for persistent telemetry and device data
- React / Vite for the dashboard
- A Python endpoint agent for network and system telemetry collection
- SQLite on the endpoint as a durable upload queue
- ML / anomaly-detection helpers for network intelligence
- Docker Compose for local multi-service deployment
- Terraform / AWS configuration as a starting point for future cloud deployment
- Automated tests and project documentation

Intended Architecture

```
Windows Host – NetSentinel Agent
|
v
(gateway ping · internet ping · DNS · HTTP · interfaces · CPU/RAM · byte counters)
|
v
FastAPI Backend API
|
v
PostgreSQL (Devices / Telemetry / Incidents / Analysis)
|
v
Intelligence Layer (Health & Anomaly Rules, ML Helpers, Incidents)
|
v
React Dashboard (Metrics / Health / Incidents / Charts)
```

2. Initial Build

The initial implementation was generated and organized as a complete multi-component project, comprising the following areas.

Backend

- FastAPI telemetry API
- Device-token authentication
- Telemetry validation

- Telemetry deduplication
- Device health scoring
- Anomaly rules
- Incident generation
- PostgreSQL integration

Endpoint Agent

A cross-platform Python agent was included with collectors for:

- Default gateway discovery
- Gateway ping
- Internet endpoint ping
- DNS resolution
- HTTP latency
- Network interfaces
- System metrics
- Network byte counters

The agent also included a durable SQLite queue for telemetry uploads.

Frontend

A React / Vite dashboard was implemented with:

- Device selection
- Health score display
- Telemetry charts
- Incident display
- Loading and error states
- Refresh functionality

Machine Learning

- Feature helpers
- Isolation Forest helpers
- Synthetic data / model training support
- Reproducible synthetic model functionality

Infrastructure

- Docker Compose
- PostgreSQL
- FastAPI container
- Frontend / nginx container
- Terraform starting point
- AWS / cloud configuration guidance

Quality and Documentation

- Tests
- CI configuration
- Architecture documentation
- Security documentation
- Cloud documentation
- Networking documentation
- Demo instructions

3. First Environment Hurdle — Node.js Was Missing

The first major local-environment issue surfaced during a check of the development toolchain. Python, pip, Docker, and Git were already available; however, the following commands initially failed because Node.js was not installed:

```
node --version
npm --version
```

This prevented the React frontend from being built.

Resolution

Node.js was installed on Windows. After installation, the version checks returned:

```
> node --version
v24.21.0

> npm --version
11.19.0
```

The installer also presented an optional "Tools for Native Modules" checkbox. These native build tools were not required for the immediate project path, so installation was completed without making them a dependency of the project workflow.

4. Python Virtual Environment

A project-local Python virtual environment was created and activated:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

The environment was then used for backend, agent, and ML dependencies, and pip was upgraded successfully.

5. Frontend Build Errors

The first frontend production build produced three TypeScript errors, involving:

1. Missing lodash declaration / type information used by Recharts
2. Missing import.meta.env typing
3. String.prototype.replaceAll not available under the configured TypeScript target

The affected files included:

- frontend/package.json
- frontend/tsconfig.json
- frontend/src/vite-env.d.ts
- frontend/src/types/lodash-recharts-compat.d.ts
- frontend/.env.example
- README.md

Fixes

- Added @types/lodash as a development dependency
- Added Vite's vite/client declaration
- Upgraded the TypeScript target/lib from ES2020 to ES2021
- Added a narrow Recharts / lodash compatibility type
- Documented the frontend API URL

The compatibility declaration was kept narrow rather than replacing lodash with a blanket untyped declaration.

Verification

The strict TypeScript check passed:

```
> npx tsc -b
tsc-exit=0
```

The frontend build subsequently completed successfully:

```
> npm run build
830 modules transformed.
built in 2.44s
```

Two build warnings were observed — Vite's deprecated CJS Node API, and a JavaScript chunk exceeding the recommended size — both of which are warnings rather than build failures.

6. npm Installation Warnings

During frontend dependency installation, npm reported:

```
2 vulnerabilities
1 moderate
1 high
```

npm suggested running `npm audit fix --force`. This was intentionally not executed during initial validation, since a forced dependency upgrade could introduce breaking changes while the application was still being stabilized. npm also reported an install-script warning involving esbuild. Dependencies nevertheless installed successfully and the frontend build passed. These warnings remain candidates for a later dependency and security-maintenance pass.

7. Backend Test Database Failure

The first backend / agent / ML test run produced:

```
5 tests collected
3 passed
2 failed
```

Both failures occurred in the API tests, when the API attempted to register a device, and reported:

```
sqlite3.OperationalError: no such table: devices
```

Root Cause

The tests constructed `TestClient(app)` without entering the FastAPI lifespan context, so application schema initialization did not run. The tests also used the normal application database session factory instead of an isolated test database fixture.

Fix

- Created a unique temporary SQLite database per test
- Created the schema through `Base.metadata.create_all()`
- Overrode the application engine / session factory during the test
- Properly entered the application lifespan
- Cleaned up the temporary database after each test
- Updated API tests to use the managed client fixture

FastAPI startup schema initialization was also migrated from the deprecated `@app.on_event("startup")` pattern to a lifespan handler.

Verification

The complete relevant suite passed:

```
5 passed
```

Two dependency-level deprecation warnings related to installed FastAPI / Starlette / httpx / AnyIO behavior remained, but there were no further project test failures. Production PostgreSQL configuration was preserved throughout.

8. Docker Compose Failure — Wrong Directory

The first attempt to start Docker Compose failed with:

```
no configuration file provided: not found
```

Cause

The terminal session had been moved one directory above the project root.

Resolution

The correct project directory was entered and the Compose file confirmed:

```
cd network-intelligence-platform  
dir docker-compose.yml
```

The file existed, and Docker Compose could then be launched correctly.

9. Docker Engine Not Running

A subsequent Docker Compose attempt failed with:

```
failed to connect to the docker API  
dockerDesktopLinuxEngine  
The system cannot find the file specified.
```

Cause

Docker Desktop's Linux engine/daemon was not running.

Resolution

Docker Desktop was started. Once the Docker daemon became available, Docker Compose started normally.

10. First Successful Docker Deployment

The project was successfully started using:

```
docker compose up --build
```

The running services were confirmed as follows.

PostgreSQL

```
database system is ready to accept connections
```

FastAPI

```
Application startup complete.  
Uvicorn running on http://0.0.0.0:8000
```

Frontend

The frontend / nginx container started successfully. docker compose ps confirmed:

```

backend    0.0.0.0:8000->8000/tcp
frontend   0.0.0.0:5173->80/tcp
postgres   0.0.0.0:5432->5432/tcp

```

The PostgreSQL container was also reported as healthy.

11. Backend API Verification

The FastAPI documentation was accessible at <http://localhost:8000/docs>. The API exposed the following routes:

```

GET /api/v1/health
GET /api/v1/ready
POST /api/v1/devices/register
POST /api/v1/telemetry
POST /api/v1/telemetry/batch
GET /api/v1/devices
GET /api/v1/devices/{device_id}
GET /api/v1/devices/{device_id}/metrics
GET /api/v1/devices/{device_id}/health
GET /api/v1/anomalies
GET /api/v1/incidents
GET /api/v1/incidents/{incident_id}

```

The health endpoint was verified:

```

GET /api/v1/health
{"status":"healthy"}

```

The unversioned route GET /health returned {"detail":"Not Found"}, which was expected, since /health was not defined — the actual health endpoint is under /api/v1/health.

12. Demo Data Integration Failure

The initial dashboard displayed "No devices yet. Register an agent or run the seed script." The synthetic seed script was executed and reported successful seeding, but GET /api/v1/devices still returned an empty array.

Root Cause

The host-side seed script targeted a local SQLite database, while the Dockerized FastAPI backend was configured against PostgreSQL — two different databases were in use, so the seed script wrote to local SQLite while the Dockerized API read from Docker PostgreSQL.

```

Seed script:  sqlite:///./netsentinel.db
Backend API:  postgresql+psycopg://netsentinel:...@postgres:5432/netsentinel

```

13. Demo Database Fix

The demo seeding workflow was redesigned so that the target database URL is always explicit.

scripts/seed_demo_data.py

- Added a `--database-url` option
- Uses the selected database URL before importing the application database module
- Prints password-redacted connection information

docker-compose.yml

A seed service was added under the demo profile, connecting to the same internal PostgreSQL service used by the backend.

README.md

Docker and host-side demo seeding instructions were documented.

Verified Host-Side Seed

The Docker PostgreSQL database was successfully seeded using:

```
python scripts\seed_demo_data.py --database-url
"postgresql+psycpg://netsentinel:netsentinel_dev@localhost:5432/netsentinel"
```

The operation inserted 80 synthetic telemetry records into the Docker PostgreSQL database.

14. Synthetic Dashboard Verification

After correcting the database mismatch, the live API returned the seeded device, and the dashboard displayed:

- Demo Wi-Fi device
- Health score
- Telemetry chart
- Open incident
- Incident classification
- Model / rule confidence

The dashboard surfaced an incident classified as UPSTREAM DEGRADATION, at 67% model/rule confidence, marked OPEN, with evidence indicating that gateway conditions were normal while internet latency had increased. The synthetic telemetry therefore successfully demonstrated the complete path from ingestion to visualization:

```
Synthetic Telemetry
|
v
PostgreSQL
|
```

```

v
FastAPI
|
v
Health / Anomaly Logic
|
v
Incident
|
v
React Dashboard

```

15. Windows Agent Registration

The project was then moved from synthetic data toward real endpoint monitoring. The agent's original implementation lacked a convenient device-registration workflow, so a registration mode was added:

```
python -m netsentinel_agent.main --register
```

The registration process returns a `device_id` and `device_token`; the token is subsequently used via the X-Device-Token header for telemetry uploads. The token is treated as a credential and should not be committed to source control or exposed in public documentation.

16. Windows Agent Configuration

The local Windows agent is configured via environment variables:

```

$env:API_BASE_URL = "http://localhost:8000"
$env:DEVICE_ID = "<device id>"
$env:DEVICE_TOKEN = "<device token>"
$env:INTERVAL_SECONDS = "10"
$env:INTERNET_TARGETS = "1.1.1.1,8.8.8.8"

```

`API_BASE_URL` is configured without the `/api/v1` suffix, since the agent appends the API path internally. The checked-in YAML configuration example is currently documentation only and is not loaded by the implementation.

17. Real Windows Device Registration

The Windows machine successfully registered with the running Docker-backed API, appearing as device ARKADESKTOP with platform windows. The device was also visible in the React dashboard's device selector alongside the synthetic Demo Wi-Fi device, confirming that the real Windows endpoint registration path functions correctly.

18. Real Windows Telemetry Collection Failure

A single agent collection cycle was executed:

```
python -m netsentinel_agent.main --once
```

The API initially returned an empty array for the ARKADESKTOP metrics endpoint, and the health endpoint returned {"detail": "no telemetry"}. At first this appeared to indicate that the agent was not collecting or uploading data; further inspection revealed a more precise cause.

19. Root Cause of the Real-Agent Failure

The agent's durable local queue showed that the agent did collect a real host measurement and did attempt the upload, but the upload failed after a retry. The measurement contained a `bytes_received` value of 3,561,180,034 — approximately 3.56 GB.

Root Cause

The PostgreSQL telemetry model stores this field as a signed 32-bit INTEGER, whose maximum value is 2,147,483,647. Since 3,561,180,034 exceeds that ceiling, PostgreSQL cannot store the cumulative byte counter in the existing column.

Current Flow

```
Windows Agent
-> Collect real telemetry      [OK]
-> Create measurement         [OK]
-> Attempt upload             [OK]
-> PostgreSQL insert          [FAIL]
-> Metrics API                []
-> Health API                 "no telemetry"
```

The Windows agent's core collection path is functioning correctly; the current blocker is the database's representation of cumulative network byte counters.

20. Correct Technical Direction for the Current Bug

The appropriate database type for cumulative network byte counters is BIGINT. The affected fields are `bytes_sent` and `bytes_received`, and the required correction is conceptually:

```
INTEGER -> BIGINT
```

The database migration must preserve existing PostgreSQL data. The change should not:

- Clamp the values
- Fake the values
- Delete telemetry
- Reset the database
- Hide the error
- Change the frontend to display fake data

The agent's upload error handling should also be improved so that HTTP/API failures are not silently swallowed.

21. Current Project Position

Infrastructure

Item	Status
Docker Desktop	Working
Docker Compose	Working
PostgreSQL container	Working
FastAPI container	Working
Frontend / nginx container	Working

Backend

Item	Status
FastAPI application	Working
Health endpoint	Working
Device registration	Working
Telemetry API	Working
PostgreSQL integration	Working
Test database isolation	Fixed
Backend tests	Passing

Frontend

Item	Status
React / Vite application	Working
TypeScript build	Passing
Production build	Passing
Dashboard rendering	Verified
Device selector	Working
Telemetry chart	Working (synthetic data)
Incident display	Working

Synthetic Pipeline

Item	Status
Demo seed	Working
PostgreSQL persistence	Working
API retrieval	Working
Health scoring	Working
Incident generation	Working
Dashboard visualization	Working

Real Windows Endpoint

Item	Status
Agent installation	Working
Device registration	Working
Device appears in dashboard	Working
Real network / system collection	Working
Upload attempt	Working
PostgreSQL persistence of large byte counters	Blocked

22. Current Known Warnings / Limitations

The project has several items that should be addressed in a later maintenance pass.

npm vulnerabilities

```
2 vulnerabilities
1 moderate
1 high
```

A forced audit fix has intentionally not been applied yet.

Vite warning

Vite reports that its CJS Node API is deprecated.

Frontend bundle size

The production build reports a JavaScript chunk larger than 500 KB after minification — an optimization concern rather than a functional failure.

Dependency deprecation warnings

The Python test suite reports dependency-level warnings involving installed FastAPI, Starlette, httpx, and AnyIO behavior.

Agent retry strategy

The agent retries failed uploads on later collection cycles but does not yet implement exponential backoff.

Agent configuration

The YAML configuration example is not currently loaded by the implementation; environment variables are used for the current workflow.

Speed test

The current agent does not perform a network speed test; upload/download Mbps values remain unset.

PostgreSQL exposure

The local Docker Compose setup currently exposes PostgreSQL on port 5432. This is acceptable for local development but should not be publicly exposed in the final cloud deployment.

Cloud deployment

No AWS resources have been deployed yet. Terraform / cloud configuration exists as a starting point, but the application is being validated locally before cloud deployment.

23. Why Cloud Deployment Has Not Happened Yet

The intended project includes cloud deployment, but deployment was deliberately deferred until the local system was validated. The current progression is:

```
Project scaffold
|
v
Local dependencies
|
v
Frontend build
|
v
Backend tests
|
v
Docker
|
v
PostgreSQL
|
v
Synthetic data
|
v
Dashboard
```

```

|
v
Real Windows agent
|
v
CURRENT BLOCKER
|
v
Fix BIGINT telemetry schema
|
v
Verify real telemetry end-to-end
|
v
Reliability testing
|
v
Security hardening
|
v
AWS deployment

```

This sequencing avoids deploying an unstable application to the cloud before the fundamental telemetry path has been validated.

24. Planned Next Steps

Immediate

4. Change cumulative network byte counters from INTEGER to BIGINT
5. Apply an idempotent PostgreSQL schema migration
6. Preserve existing Docker PostgreSQL data
7. Improve agent upload error reporting
8. Run one real Windows collection cycle
9. Verify ARKADESKTOP telemetry through the live API
10. Verify health calculation
11. Verify PostgreSQL persistence
12. Verify dashboard visualization

Reliability Testing

After real telemetry is confirmed working:

- Stop the backend/API
- Continue running the agent
- Confirm telemetry enters the local SQLite queue
- Restart the API
- Confirm queued telemetry is retried
- Confirm it reaches PostgreSQL
- Test repeated failures

- Verify retry limits
- Test malformed API responses
- Test network interruption/recovery

Target architecture for this test:

```
API DOWN
|
v
Windows Agent
|
v
SQLite Queue
|
v
PENDING / FAILED
|
v
API returns
|
v
Retry
|
v
FastAPI
|
v
PostgreSQL
```

Security Hardening

Planned review areas include:

- Device-token handling
- Secrets
- Environment variables
- CORS
- API authentication
- Input validation
- Rate limiting
- Database exposure
- Container privileges
- Production configuration

Cloud Deployment

Following local validation, planned steps include:

- Containerize production services
- Deploy the API
- Deploy PostgreSQL using an appropriate managed service
- Configure HTTPS

- Configure secrets
- Deploy the dashboard
- Configure networking / security groups
- Deploy the Windows agent against the cloud API
- Add monitoring and logging
- Add Terraform automation

25. Development Philosophy Learned From the Build

One of the major lessons from this project has been the difference between "the code builds" and "the system actually works." Several issues only became visible through end-to-end testing:

```
Frontend compilation issue
|
v
Backend test database issue
|
v
Docker environment issue
|
v
Host-vs-container database mismatch
|
v
Real-agent telemetry persistence issue
```

Each layer passed only after the previous layer was tested independently. The project therefore follows an incremental validation strategy:

```
Compile
|
v
Unit / API tests
|
v
Container startup
|
v
API health
|
v
Synthetic data
|
v
Dashboard
|
v
Real endpoint registration
|
v
Real telemetry
|
v
```

```

Failure / recovery testing
|
v
Cloud deployment

```

This approach reduces the risk of confusing an environment problem, integration problem, database problem, or application bug with one another.

26. Current Status Summary

Summary

NetSentinel is currently a functioning local Dockerized network-intelligence prototype with a verified synthetic end-to-end pipeline and a registered real Windows monitoring endpoint.

The remaining immediate blocker:

```

PostgreSQL INTEGER
|
v
Cannot store cumulative byte counter
|
v
Change to BIGINT

```

Once that schema issue is resolved and verified, the next milestone is to demonstrate the full real-data path:

```

Real Windows PC
|
v
Real network telemetry
|
v
NetSentinel agent
|
v
FastAPI
|
v
PostgreSQL
|
v
Anomaly / health engine
|
v
Incident
|
v
React dashboard

```

Only after that pipeline is stable should the system be promoted to its planned cloud deployment.

27. Repository Hygiene Before Publishing

Before publishing this report or the repository publicly, it should be verified that the repository does not contain:

- Device tokens
- API secrets
- Database passwords intended for production
- .env files containing credentials
- netsentinel-agent.db
- netsentinel.db
- PostgreSQL data volumes
- Private machine-specific credentials

Development-only credentials should remain documented as examples only. .env.example files should be used for configuration templates, and real .env files should be kept ignored by Git. The Windows device token generated during development must not be included in this report or committed to the repository.

Final Milestone — Overall Status

Area	Status
Local platform	Working
Synthetic end-to-end pipeline	Verified
Real Windows registration	Verified
Real Windows telemetry collection	Verified
Real Windows telemetry persistence	Blocked (BIGINT schema issue)
Cloud deployment	Not started
Production hardening	Pending

Next Technical Milestone

Migrate network byte counters to PostgreSQL BIGINT, verify real telemetry persistence, and complete the real endpoint → API → database → dashboard path.